

運用 ADK、MCP Toolbox 和 AlloyDB 打造運動用品店 AI 助理

來源：<https://codelabs.developers.google.com/codelabs/sports-agent-adk-mcp-alloydb?hl=zh-tw>

步驟數：13

在這個程式碼研究室中，您將瞭解如何建立 AI 服務專員，協助您處理運動零售業務中的任務、取得推薦內容、尋找商店或下單。您將使用 Agent Development Kit (ADK) 建構代理，並使用 MCP Toolbox for Databases 將代理連結至 AlloyDB

目錄

1. [1. 簡介](#)
2. [2. 事前準備](#)
3. [3. 建立 AlloyDB 執行個體](#)
4. [4. 載入資料庫](#)
5. [5. 設定授權服務](#)
6. [6. 設定 MCP Toolbox for Databases](#)
7. [7. 以 ADK 建構的代理](#)
8. [8. 部署前端](#)
9. [9. 執行代理程式](#)
10. [10. 結果](#)
11. [11. \(選用\) 測試 AlloyDB AI 自然語言轉 SQL 功能](#)
12. [12. 清除](#)
13. [13. 恭喜](#)

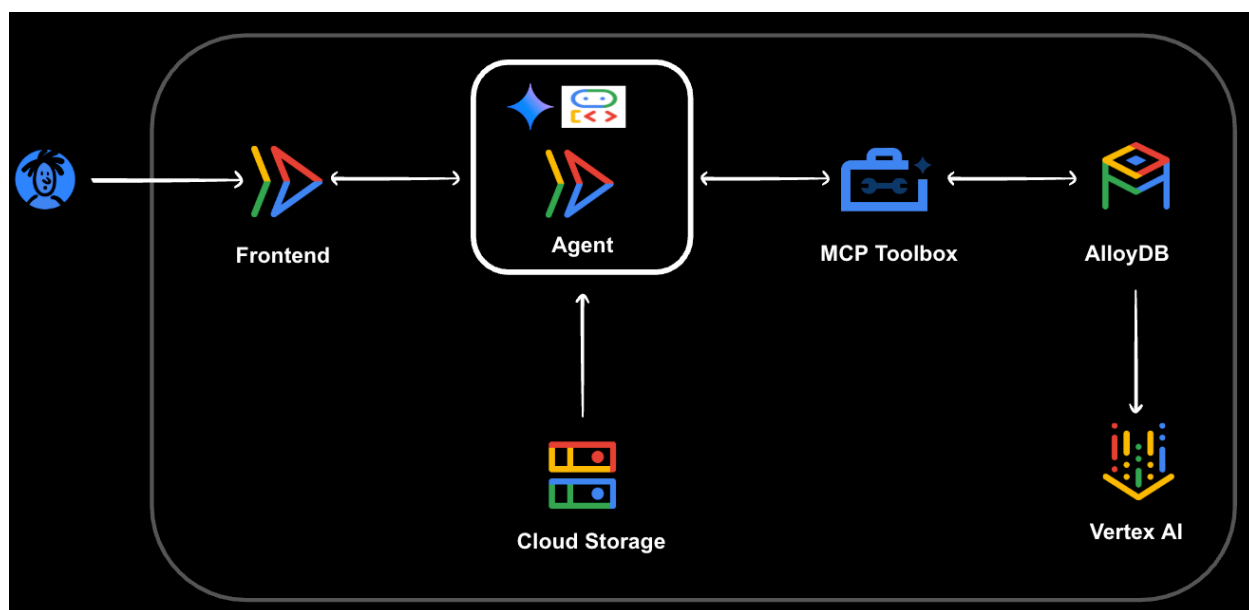
步驟 1 · 約 0 分鐘

1. 簡介

建構項目

在本程式碼研究室中，您將瞭解如何建構運動用品店代理 **AI 助理**。這款新一代代理 AI 應用程式採用 **ADK**、**MCP Toolbox** 和 **AlloyDB**，可協助使用者完成各種工作，包括：

- 使用自然語言搜尋產品。
- 尋找附近商店以購買推薦產品。
- 下新單。
- 查看現有訂單狀態。
- 更新訂單，加入偏好的運送方式。



課程內容

- 佈建及填入 PostgreSQL 適用的 AlloyDB 資料庫。
- 使用 AlloyDB for PostgreSQL 執行個體設定 MCP Toolbox for Databases。
- 使用 Agent Development Kit (ADK) 設計及開發 AI 代理，協助處理運動用品店查詢。
- 在雲端環境中測試代理程式和 MCP Toolbox for Databases。
- 運用 AlloyDB 的進階查詢功能，提供智慧型代理的回覆。

軟硬體需求

如果要完成本程式碼研究室，請務必符合以下條件：

- Chrome 網路瀏覽器。
- Gmail 帳戶。
- 已啟用計費功能的 Google Cloud 專案。

本程式碼研究室適合各種程度的開發人員 (包括初學者)。

步驟 2 · 約 0 分鐘

2. 事前準備

本節將逐步說明在 Google Cloud 專案中進行初始設定的必要步驟，完成後即可開始建構 Sports Shop Agent AI 助理。

建立專案

1. 在 [Google Cloud 控制台](#) 的專案選取器頁面中，選取或建立 Google Cloud 專案。
2. 確認 Cloud 專案已啟用計費功能。瞭解如何[檢查專案是否已啟用計費功能](#)。
3. 按一下這個[連結](#)，啟動 Cloud Shell。如要在 Cloud Shell 終端機 (用於執行雲端指令) 和編輯器 (用於建構專案) 之間切換，請點選 Cloud Shell 中的對應按鈕。



4. 連至 Cloud Shell 後，請使用下列指令確認驗證已完成，專案也已設為獲派的專案 ID：

```
gcloud auth list
```

5. 在 Cloud Shell 中執行下列指令，確認 gcloud 指令已瞭解您的專案。

```
gcloud config list project
```

6. 設定 PROJECT_ID 變數，請使用下列指令：

```
export PROJECT_ID=[YOUR_PROJECT_ID]
```

```
gcloud config set project $PROJECT_ID
```

7. 執行下列指令，啟用下列 API：

```
gcloud services enable alloydb.googleapis.com \  
compute.googleapis.com \  
cloudresourcemanager.googleapis.com \  
servicenetworking.googleapis.com \  
vpcaccess.googleapis.com \  
aiplatform.googleapis.com
```

步驟 3 · 約 0 分鐘

3. 建立 AlloyDB 執行個體

在本節中，您將設定 AlloyDB 資料庫叢集和執行個體，並為 AI 代理程式設定必要的網路和權限。

首先，請在 Cloud Shell 終端機中執行下列指令，建立 AlloyDB 叢集：

```
gcloud alloydb clusters create alloydb-cluster \  
  --password=alloydb\  
  --network=default \  
  --region=us-central1 \  
  --database-version=POSTGRES_16
```

AlloyDB 採用私人 IP 連線，確保存取安全無虞，並提供高效能。您必須在 VPC 內分配私人 IP 範圍，供 Google 用於與 Google 代管服務網路基礎架構的服務對等互連連線。執行下列指令：

```
gcloud compute addresses create peering-range-for-alloydb \  
  --global \  
  --purpose=VPC_PEERING \  
  --prefix-length=16 \  
  --description="Automatically allocated IP range for service networking" \  
  --network=default
```

接著，建立 VPC 服務對等互連連線。這樣一來，Google Cloud 虛擬私有雲 (VPC) 網路就能與 Google 的受管理服務 (包括 AlloyDB) 安全地進行私密通訊。執行下列指令：

```
gcloud services vpc-peerings connect \  
  --service=servicenetworking.googleapis.com \  
  --ranges=peering-range-for-alloydb \  
  --network=default
```

現在，請在 AlloyDB 叢集中建立主要執行個體。這是應用程式實際連線的資料庫端點。執行下列指令來建立 AlloyDB 執行個體：

```
gcloud alloydb instances create alloydb-inst \  
  --instance-type=PRIMARY \  
  --cpu-count=2 \  
  --region=us-central1 \  
  --cluster=alloydb-cluster \  
  --availability-type=ZONAL \  
  --ssl-mode=ALLOW_UNENCRYPTED_AND_ENCRYPTED
```

注意：建立執行個體約需 **10 分鐘**。請等待這項作業完成後再繼續操作。

啟用 Vertex AI 整合功能

如要讓 AlloyDB 執行向量搜尋查詢 (這類查詢是語意搜尋等 AI 功能的必要條件)，並呼叫部署在 Vertex AI 中的模型，您需要將 Vertex AI 權限授予 AlloyDB 服務代理程式。

首先，請先擷取 Google Cloud 專案編號，因為 IAM 繫結需要這個編號。

```
PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID --format="value(projectNumber)")
gcloud projects describe $PROJECT_ID --format="value(projectNumber)"
```

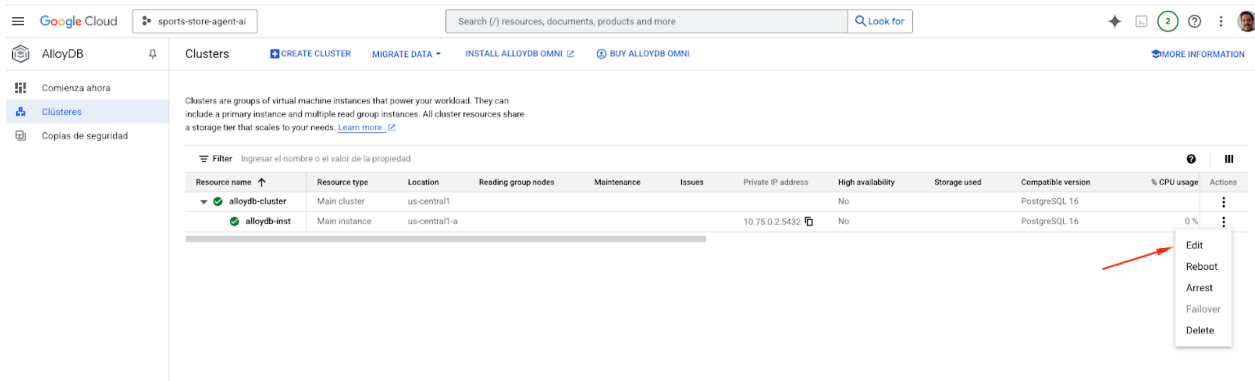
然後將 Vertex AI 權限授予 AlloyDB 服務代理：

```
gcloud projects add-iam-policy-binding $PROJECT_ID \
--member="serviceAccount:service-$PROJECT_NUMBER@gcp-sa-alloydb.iam.gserviceaccount.com" \
--role="roles/aiplatform.user"
```

啟用公開 IP

為準備執行後續步驟，我們將在 AlloyDB 執行個體中啟用公開 IP 連線。

在控制台畫面頂端中間的搜尋欄中輸入「AlloyDB」，然後編輯並前往「Public IP connectivity」部分。勾選「啟用公開 IP」核取方塊，然後輸入 Cloud Shell 機器的 IP 位址。



如要取得 Cloud Shell 機器的 IP，請前往 Cloud Shell 終端機，然後輸入「ifconfig | grep -A 1 eth0」指令。從結果中，將最後 2 位數替換為 0.0，遮罩大小為「/16」。例如「XX.XX.0.0/16」，其中 XX 為數字。

將這個 IP 貼到編輯執行個體頁面的「授權外部網路」文字方塊中。

Public IP connectivity

☒ Enable public IP

Connect your applications and clients over the public Internet. May require additional configuration to ensure a secure connection. [Learn more](#) 



For the most secure connectivity, use AlloyDB connectors or AlloyDB Auth Proxy. Alternatively, make sure that your client IPs or networks are added as authorised external networks. [Learn more](#) 



Public IP requires password complexity to be enabled. We will be enabling the following flag:
`password.enforce_complexity`

Authorised external networks

Networks



Enter a range with a mask size between /16 and /32 bits.

注意：更新作業最多可能需要 3 分鐘

步驟 4 · 約 0 分鐘

4. 載入資料庫

建立商店資料庫

現在可以建立資料庫，並載入運動用品商店的初始資料。

如要讓 `psql` 從 Cloud Shell 連線至私人 AlloyDB 執行個體，請使用 AlloyDB Auth Proxy。這項公用程式會安全地將連線通道連至資料庫。(請參閱「[AlloyDB Auth Proxy](#)」)

使用下列指令下載 AlloyDB Auth Proxy：

```
wget https://storage.googleapis.com/alloydb-auth-proxy/v1.13.6/alloydb-auth-proxy.linux.amd64
-O alloydb-auth-proxy
```

將其設為可執行：

```
chmod +x alloydb-auth-proxy
```

在第一個 Cloud Shell 終端機視窗中執行這項指令。Proxy 會在背景執行並轉送連線。

```
nohup ./alloydb-auth-proxy "projects/$PROJECT_ID/locations/us-central1/clusters/alloydb-
cluster/instances/alloydb-inst" --public-ip &
```

使用 `psql` 連線至 AlloyDB 執行個體：

```
psql -h 127.0.0.1 -U postgres
```

注意：系統提示時，請輸入您在建立叢集時為 `postgres` 使用者設定的密碼 (如果您直接按照文件操作，密碼為 `alloydb`)。

並為應用程式建立商店資料庫 (逐一執行指令)：

```
CREATE DATABASE store;
\c store
exit
```

原始碼

現在，請複製程式碼研究室的原始碼存放區。請先確認目前的位置是主目錄或合適的位置，再複製並執行下列指令：

```
git clone https://github.com/mtoscano84/sports-agent-adk-mcp-alloydb.git
```

填入資料庫

前往複製專案的 `data` 資料夾，存取資料庫傾印檔案。

```
cd sports-agent-adk-mcp-alloydb/data
```


接著，使用存放區中的 `store_backup.sql` 檔案，將範例資料集匯入 `store` 資料庫。

```
psql -h 127.0.0.1 -U postgres -d store -f store_backup.sql
```

注意：匯入期間可能會看到一些 WARNING 和 ERROR 訊息，請放心忽略。如果傾印包含完整結構定義，這些通常與權限或現有物件有關。您會發現一些可忽略的警告和錯誤

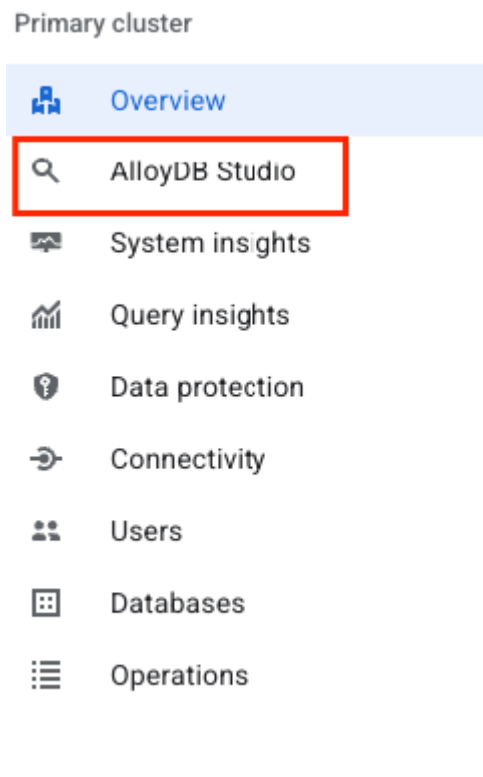
步驟 5 · 約 0 分鐘

5. 設定授權服務

在本節中，您將為應用程式設定授權服務。這項服務對於確保存取安全，以及防範 AI 代理程式中的[提示詞注入](#)漏洞至關重要。

首先，請在 `store` 資料庫的 `users` 表格中新增範例使用者。這個使用者會用於應用程式中的驗證。

前往控制台並導覽至 AlloyDB，選取主要執行個體，然後選取 AlloyDB Studio：



系統提示時，請使用您設定叢集時建立的憑證登入 AlloyDB Studio：

- 使用者名稱：「postgres」
- 資料庫：「store」
- 密碼：「alloydb」

在 SQL 編輯器中執行 INSERT 陳述式，將使用者新增至資料庫。變更姓名、姓氏和電子郵件地址。

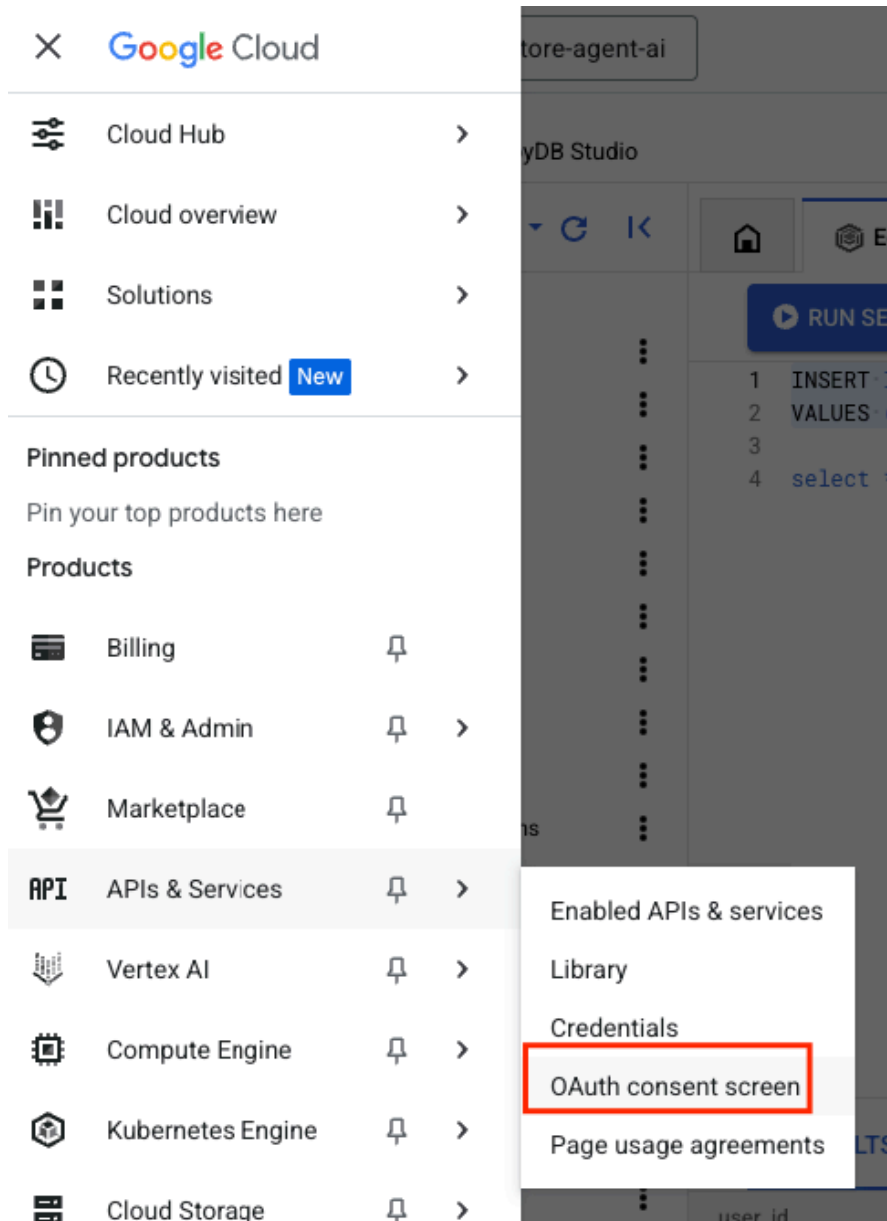
重要事項：

- 保留範例中的 LOCATION
- 使用您在 Google Cloud 控制台註冊時的電子郵件地址

```
INSERT INTO users (user_id, first_name, last_name, Address, city, postal_code, location, email)
VALUES (10, 'John', 'Doe', 'Carrer Muntaner 39', 'Barcelona', '08019',
'0101000020E61000008AAE0B3F38B144401FBB0B9414780140', 'john.doe@example.com');
```

接著，請為專案設定 OAuth 同意畫面。當應用程式要求存取使用者的 Google 帳戶時，系統會向使用者顯示這個畫面，並定義應用程式的品牌。

在控制台中，依序前往「API 和服務」、「Google OAuth 同意畫面」：



提供下列資訊，建立應用程式的品牌：

- 應用程式名稱：「Sports Shopping Agent AI」
- 使用者支援電子郵件地址：「YOUR_EMAIL」
- 目標對象：「外部」
- 聯絡資訊：「YOUR_EMAIL」

現在，您要建立前端應用程式將使用的 OAuth 用戶端 ID，以便透過 Google 驗證使用者身分。

首先，請確認您有 Google Cloud 專案編號。這是正確設定重新導向 URI 的必要條件。在 Cloud Shell 終端機中執行下列指令：

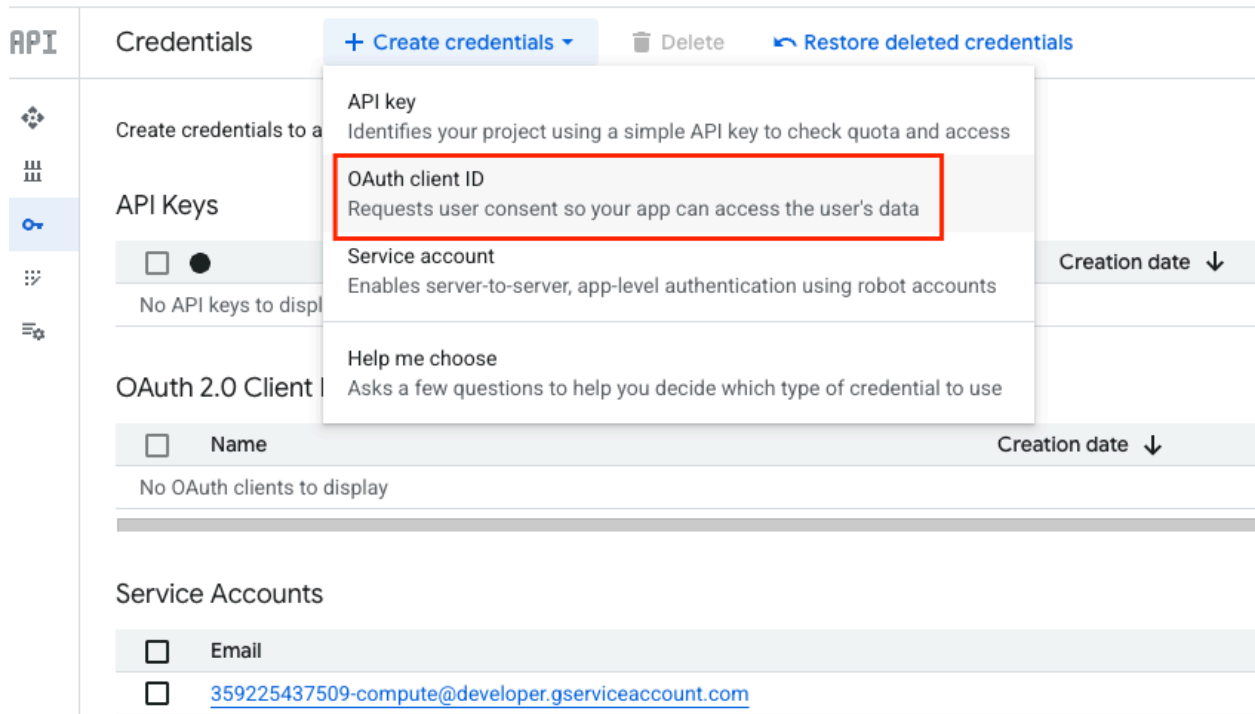
如果這個 Cloud Shell 終端機視窗中未設定 PROJECT_ID 變數，請執行下列指令：

```
export PROJECT_ID=[YOUR_PROJECT_ID]
```

然後使用下列指令取得 PROJECT_NUMBER：

```
gcloud projects describe $PROJECT_ID --format="value(projectNumber)"
```

然後依序前往「API 和服務」->「憑證」->「建立憑證」->「OAuth 用戶端 ID」



請使用下列資訊建立憑證：

- 應用程式類型：「網頁應用程式」
- 名稱：「Sports Shopping Agent AI App」(運動購物代理 AI 應用程式)

已授權的 JavaScript 來源：

- 網址 1：[https://finn-frontend-\[YOUR_PROJECT_NUMBER\].us-central1.run.app](https://finn-frontend-[YOUR_PROJECT_NUMBER].us-central1.run.app)

已授權的重新導向 URI：

- 網址 1：[https://finn-frontend-\[YOUR_PROJECT_NUMBER\].us-central1.run.app](https://finn-frontend-[YOUR_PROJECT_NUMBER].us-central1.run.app)

注意：[https://finn-frontend-\[YOUR_PROJECT_NUMBER\].us-central1.run.app](https://finn-frontend-[YOUR_PROJECT_NUMBER].us-central1.run.app) 是前端應用程式的預期部署網址，稍後會在程式碼研究室中設定。請務必將 [YOUR_PROJECT_NUMBER] 替換為您複製的實際編號。

[←](#) Create OAuth Client ID

Application type *
Web application ▼

Name *
Sports Shopping Agent AI App

The name of your OAuth 2.0 client. This name is only used to identify the client in the console and will not be displayed to end users.



The domains with the URIs you add below will automatically be added to your [OAuth consent screen](#) as [authorized domains](#). [↗](#)

Authorized JavaScript Sources [?](#)

For use with browser requests

URI 1 *
`https://finn-frontend-992680354553.us-central1.run.app`

[+ Add URI](#)

Authorized Redirect URIs [?](#)

For use with requests from a web server

URI 1 *
`https://finn-frontend-992680354553.us-central1.run.app`

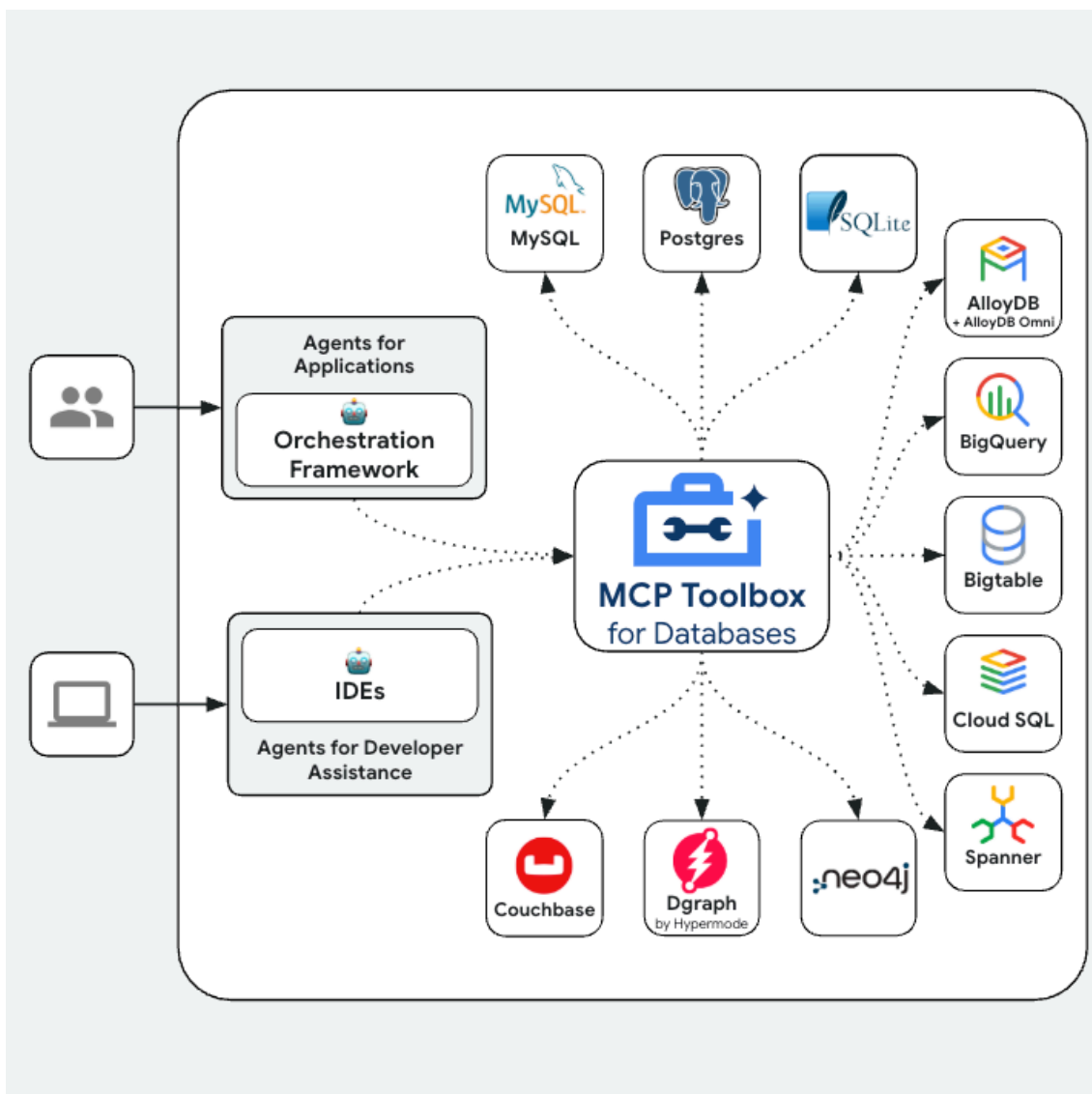
[+ Add URI](#)

重要事項：建立完成後，系統會顯示彈出式視窗，內含 **OAuth 用戶端 ID**，有時也會顯示用戶端密鑰。將 **OAuth 用戶端 ID** 儲存在安全的地方，因為您會在後續步驟中設定前端時需要這個 ID。

6. 設定 MCP Toolbox for Databases

Toolbox 位於應用程式的協調架構和資料庫之間，提供用於修改、發布或叫用工具的控制層。這項功能提供集中式位置來儲存及更新工具，方便您管理工具，並在代理程式和應用程式之間共用工具，以及更新工具，不必重新部署應用程式。

MCP Toolbox for Databases 支援的資料庫之一是 AlloyDB，而我們已在上一節中佈建該資料庫，因此請繼續設定 Toolbox。



首先，您會在 Cloud Shell 殼層環境中在本機設定 MCP Toolbox 伺服器，以驗證其功能。

1. 在 Cloud Shell 終端機中，前往複製專案存放區中的 `toolbox` 資料夾：

```
cd sports-agent-adk-mcp-alloydb/src/toolbox
```

2. 執行下列指令，下載 Toolbox 二進位檔並授予執行權限：

```
# see releases page for other versions
export VERSION=0.16.0

curl -O https://storage.googleapis.com/mcp-toolbox-for-
databases/v$VERSION/linux/amd64/toolbox

chmod +x toolbox
```

注意：這裡指定的是 0.16.0 版。在正式版環境中，請務必驗證並使用[工具箱發布頁面](#)的最新穩定版本。

3. 前往 Cloud Shell 編輯器 (點選「編輯器」圖示即可從終端機切換)。



在同一個 `sports-agent-adk-mcp-alloydb/src/toolbox` 目錄中，您會找到名為 `tools.yaml` 的檔案。開啟這個檔案，然後使用先前步驟中的 OAuth 用戶端 ID 和 Google Cloud 專案 ID 更新預留位置。

```
authServices:
  google_signin:
    kind: google
    clientId: [REDACTED]
sources:
  my-alloydb-pg-source:
    kind: alloydb-postgres
    project: sports-store-agent-ai
    region: us-central1
    cluster: alloydb-cluster
    instance: alloydb-inst
    database: store
    user: postgres
    password: alloydb
```

讓我們瞭解 tools.yaml。

來源代表工具可互動的不同資料來源。來源代表工具可互動的資料來源。您可以在 `tools.yaml` 檔案的來源區段中，將來源定義為對應。一般來說，來源設定會包含連線及與資料庫互動所需的任何資訊。

工具會定義代理程式可執行的動作，例如讀取及寫入來源。工具代表代理程式可執行的動作，例如執行 SQL 陳述式。您可以在 `tools.yaml` 檔案的 `tools` 區段中，將工具定義為對應。一般來說，工具需要來源才能執行動作。

如要進一步瞭解如何設定 `tools.yaml`，請參閱這份[說明文件](#)。

執行 MCP Toolbox for Databases 伺服器

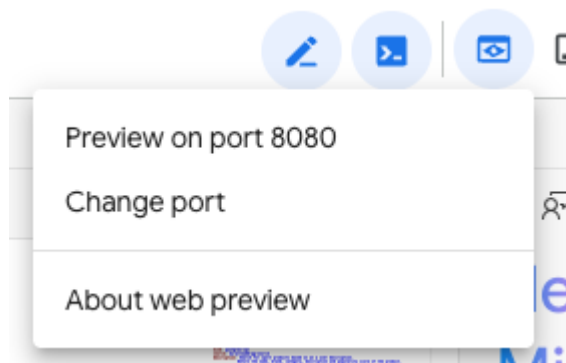
執行下列指令 (從 `mcp-toolbox` 資料夾) 啟動伺服器：

```
./toolbox --tools-file "tools.yaml"
```

現在，如果您在雲端以網頁預覽模式開啟伺服器，應該會看到 Toolbox 伺服器已啟動並執行，且包含應用程式的所有工具。

MCP Toolbox 伺服器預設會在通訊埠 5000 上執行。我們將使用 Cloud Shell 測試這項功能。

在 Cloud Shell 中點選「網頁預覽」，如下所示：



按一下「變更通訊埠」，將通訊埠設為 5000 (如下所示)，然後按一下「變更並預覽」。

Change Preview Port

Port Number *

5000

Cancel Change and Preview

這應該會產生以下輸出內容：

 Hello, World! 

MCP Toolkit for Databases 說明如何使用 Python SDK 驗證及測試工具，相關文件請參閱[這裡](#)。我們將略過這個步驟，直接進入下一節的 Agent Development Kit (ADK)，並使用這些工具。

將 Toolbox 部署至 Cloud Run

如要讓 Toolbox 伺服器成為可與其他應用程式和 AI 代理程式整合的公開端點，請將其部署至 Cloud Run。如需在 Cloud Run 上代管工具箱的詳細操作說明，請參閱[這篇文章](#)。

返回 Cloud Shell 終端機，然後前往 toolbox 資料夾：

```
cd sports-agent-adk-mcp-alloydb/src/toolbox
```

確認 `PROJECT_ID` 環境變數已設為您的 Google Cloud 專案 ID。

```
export PROJECT_ID=$PROJECT_ID
```

接著，請確認專案中已啟用下列 Google Cloud 服務。


```
gcloud services enable run.googleapis.com \
    cloudbuild.googleapis.com \
    artifactregistry.googleapis.com \
    iam.googleapis.com \
    secretmanager.googleapis.com
```

我們將建立個別的服務帳戶，做為要在 Google Cloud Run 部署的 Toolbox 服務身分。我們也會確保這個服務帳戶具備正確角色，也就是有權存取 Secret Manager 並與 AlloyDB 通訊。

```
gcloud iam service-accounts create toolbox-identity

gcloud projects add-iam-policy-binding $PROJECT_ID \
    --member serviceAccount:toolbox-identity@$PROJECT_ID.iam.gserviceaccount.com \
    --role roles/secretmanager.secretAccessor

gcloud projects add-iam-policy-binding $PROJECT_ID \
    --member='serviceAccount:toolbox-identity@$PROJECT_ID.iam.gserviceaccount.com' \
    --role='roles/alloydb.client'

gcloud projects add-iam-policy-binding $PROJECT_ID \
    --member='serviceAccount:toolbox-identity@$PROJECT_ID.iam.gserviceaccount.com' \
    --role='roles/serviceusage.serviceUsageConsumer'
```

接著，您會將 tools.yaml 檔案上傳為密碼，由於我們必須在 Cloud Run 中安裝 Toolbox，因此會使用 Toolbox 的最新容器映像檔，並在 IMAGE 變數中設定該映像檔。

```
gcloud secrets create tools --data-file=tools.yaml

export IMAGE=us-central1-docker.pkg.dev/database-toolbox/toolbox/toolbox:latest
```

最後，使用下列指令將 Toolbox 伺服器部署至 Cloud Run。這個指令會將應用程式容器化、設定服務帳戶、插入密鑰，並公開密鑰：

```
gcloud run deploy toolbox \
    --image $IMAGE \
    --service-account toolbox-identity \
    --region us-central1 \
    --set-secrets "/app/tools.yaml=tools:latest" \
    --args="--tools_file=/app/tools.yaml","--address=0.0.0.0","--port=8080" \
    --allow-unauthenticated
```

這會開始使用我們設定的 tools.yaml，將 Toolbox 伺服器部署至 Cloud Run。部署成功後，您應該會看到類似以下的訊息：

```
Deploying container to Cloud Run service [toolbox] in project [sports-store-agent-ai] region [us-central1]
OK Deploying... Done.
  OK Creating Revision...
  OK Routing traffic...
  OK Setting IAM Policy...
Done.
Service [toolbox] revision [toolbox-00002-dn2] has been deployed and is serving 100 percent of traffic.
Service URL: https://toolbox-[YOUR_PROJECT_NUMBER].us-central1.run.app
```

您現在可以在瀏覽器中開啟上方列出的服務網址。應該會顯示先前看到的「Hello World」訊息。此外，您也可以造訪下列網址，查看可用的工具：

```
https://toolbox-[YOUR_PROJECT_NUMBER].us-central1.run.app/api/toolset
```

您也可以從 Google Cloud 控制台前往 [Cloud Run](#)，即可在 Cloud Run 的服務清單中看到 Toolbox 服務。

7. 以 ADK 建構的代理

在本節中，您將使用 Agent Development Kit (ADK) 建構 AI 代理，並將其部署至 Cloud Run。

首先，請在專案中啟用必要 API，以便在 Cloud Run 上建構及部署 Agent，並與 Artifact Registry 和 Cloud Storage 互動。在 Cloud Shell 終端機中執行下列指令：

```
gcloud services enable artifactregistry.googleapis.com \
    cloudbuild.googleapis.com \
    run.googleapis.com \
    storage.googleapis.com
```

接著，我們會將必要權限指派給專案中的預設 Compute 服務帳戶。首先，請在 Cloud Shell 終端機中執行下列指令，取得 PROJECT_NUMBER：

```
PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID --format="value(projectNumber)")
```

將權限指派給預設的 Compute 服務帳戶：

```
# Grant Cloud Run service account access to GCS
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:$PROJECT_NUMBER-compute@developer.gserviceaccount.com" \
  --role="roles/storage.admin"

gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:$PROJECT_NUMBER-compute@developer.gserviceaccount.com" \
  --role="roles/run.admin"

gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:$PROJECT_NUMBER-compute@developer.gserviceaccount.com" \
  --role="roles/artifactregistry.writer"

gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:$PROJECT_NUMBER@cloudbuild.gserviceaccount.com" \
  --role="roles/artifactregistry.repoAdmin"

# Grant Vertex AI User role to the service account
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:$PROJECT_NUMBER-compute@developer.gserviceaccount.com" \
  --role="roles/aiplatform.user"

# Grant Vertex AI Model User role to the service account
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:$PROJECT_NUMBER-compute@developer.gserviceaccount.com" \
  --role="roles/aiplatform.modelUser"
```

將 Agent 連線至工具

我們將 Agent 連線至 Tools。在 ADK 架構中，工具代表賦予 AI 代理的特定能力，使其能執行各種動作並與世界互動，而不侷限於核心的文字生成和推論能力。

在本例中，我們現在要為代理提供在 MCP Toolbox for Databases 中設定的工具。

使用 Cloud Shell 編輯器前往 `sports-agent-adk-mcp-alloydb/src/backend/`，並使用下列程式碼編輯「`finn_agent.py`」檔案。請注意，我們使用的是上一步部署的 MCP ToolBox 伺服器 Cloud Run 服務網址：

```
async def process_message(message: str, history: list, session_id: str, user_id: str, id_token: str = None):
    # Create session service per request (or use a shared one if you know it's safe)
    session_service = InMemorySessionService()

    toolbox = ToolboxToolset(
        server_url="https://toolbox-[YOUR_PROJECT_NUMBER].us-central1.run.app",
        toolset_name="my-toolset",
        auth_token_getters={"google_signin": get_auth_token}
    )
```

在 Cloud Run 部署代理

最後，您會將設定好的 AI 代理程式部署至 Cloud Run，透過 HTTP 端點存取。

首先，請在 Artifact Registry 中建立 Docker 存放區，用於儲存 Agent 的容器映像檔。在 Cloud Shell 中執行下列指令：

```
gcloud artifacts repositories create finn-agent-images \
  --repository-format=docker \
  --location=us-central1 \
  --project=$PROJECT_ID \
  --description="Repository for finn-agent images"
```

接著，使用 Cloud Build 建構 Agent 的 Docker 映像檔。在複製專案的根目錄 (`sports-agent-adk-mcp-alloydb/`) 中執行下列指令：

```
gcloud builds submit src/backend/ --tag us-central1-docker.pkg.dev/$PROJECT_ID/finn-agent-images/finn-agent
```

現在請部署 Agent 服務。這個指令會建立 Cloud Run 服務、從 Artifact Registry 提取映像檔，並設定環境變數

```
gcloud run deploy finn-agent \
  --image us-central1-docker.pkg.dev/$PROJECT_ID/finn-agent-images/finn-agent \
  --platform managed \
  --allow-unauthenticated \
  --region us-central1 \
  --project $PROJECT_ID --set-env-
vars="GOOGLE_CLOUD_PROJECT=$PROJECT_ID,GOOGLE_CLOUD_LOCATION=us-
central1,GOOGLE_GENAI_USE_VERTEXAI=TRUE"
```

注意：我們會動態設定環境變數，包括 `GOOGLE_CLOUD_PROJECT` (使用殼層變數 `$PROJECT_ID`)

您應該會看到類似以下的輸出內容，表示 Agent 部署成功：

```
Deploying container to Cloud Run service [finn-agent] in project [sports-store-agent-ai]
region [us-central1]
OK Deploying... Done.
  OK Creating Revision...
  OK Routing traffic...
  OK Setting IAM Policy...
Done.
Service [finn-agent] revision [finn-agent-00005-476] has been deployed and is serving 100
percent of traffic.
Service URL: https://finn-agent-359225437509.us-central1.run.app
```

最後，請從 Cloud Shell 終端機執行下列 `curl` 指令，測試代理程式：

```
curl -X POST \
  -H "Content-Type: application/json" \
  -d '{"message": "Hello"}' \
  https://finn-agent-[YOUR_PROJECT_NUMBER].us-central1.run.app/chat
```

輸出內容應會類似以下內容：

「你好！我是 Finn，你的 AI 運動購物助理。我可以協助你尋找運動產品、裝備和器材。請問你今天需要哪方面的協助？」

此時，您已成功驗證 AlloyDB、MCP Toolbox 和使用 ADK 建構的 Agent 部署作業。

步驟 8 · 約 0 分鐘

8. 部署前端

在本節中，您將在 Cloud Run 上部署 AI 助理的對話式使用者介面。這個前端是使用 React 和 JavaScript 建構而成。

部署前，您需要使用已部署的 Agent 網址和 OAuth 用戶端 ID，更新前端的原始碼。

使用 Cloud Shell 編輯器前往 `sports-agent-adk-mcp-alloydb/src/frontend/src/pages/`，然後開啟 `Home.jsx` 檔案。您需要更新 Agent 的 Cloud Run 服務網址預留位置。然後換成上一個步驟中代理的 Cloud Run 服務網址 (例如 `https://finn-agent-[YOUR_PROJECT_NUMBER].us-central1.run.app`)。

```
import React, { useState, useEffect, useRef } from 'react';
import backgroundImage from '../images/background.jpeg';
import { MapContainer, TileLayer, Marker, Popup } from 'react-leaflet';
import 'leaflet/dist/leaflet.css';
import L from 'leaflet';
import StoreMap from '../components/StoreMap';

const BACKEND_URL = 'https://finn-agent-[YOUR_PROJECT_NUMBER].us-central1.run.app';
```

接著，前往 `sports-agent-adk-mcp-alloydb/src/frontend/src/components/` 並開啟 `GoogleSignInButton.jsx` 檔案。您將使用「授權服務設定」一節中取得的 OAuth 用戶端 ID 更新這個檔案：

```
import React, { useEffect } from "react";

export default function GoogleSignInButton({ onSignIn }) {
  useEffect(() => {
    // Load Google Sign-In script
    const script = document.createElement('script');
    script.src = 'https://accounts.google.com/gsi/client';
    script.async = true;
    script.defer = true;
    document.body.appendChild(script);

    script.onload = () => {
      if (window.google && window.google.accounts) {
        window.google.accounts.id.initialize({
          client_id: "REDACTED",
          callback: (response) => {
            // Store the ID token
            onSignIn(response.credential);
          },
        });
        window.google.accounts.id.renderButton(
          document.getElementById("google-signin-btn"),
          { theme: "outline", size: "large" }
        );
      }
    };
  });
}
```

在 Cloud Run 上部署前端

前端應用程式設定完成後，即可部署至 Cloud Run。

在 Cloud Shell 終端機中，從根目錄 (`sports-agent-adk-mcp-alloydb/`) 執行下列指令，在 Artifact Registry 中為前端映像檔建立 Docker 存放區。

```
gcloud artifacts repositories create finn-frontend-images \
  --repository-format=docker \
  --location=us-central1 \
  --project=$PROJECT_ID \
  --description="Repository for finn-frontend images"
```

接著，使用 Cloud Build 建構前端應用程式的 Docker 映像檔。從專案的根目錄執行下列指令：

```
gcloud builds submit src/frontend/ --tag us-central1-docker.pkg.dev/$PROJECT_ID/finn-frontend-images/finn-frontend
```

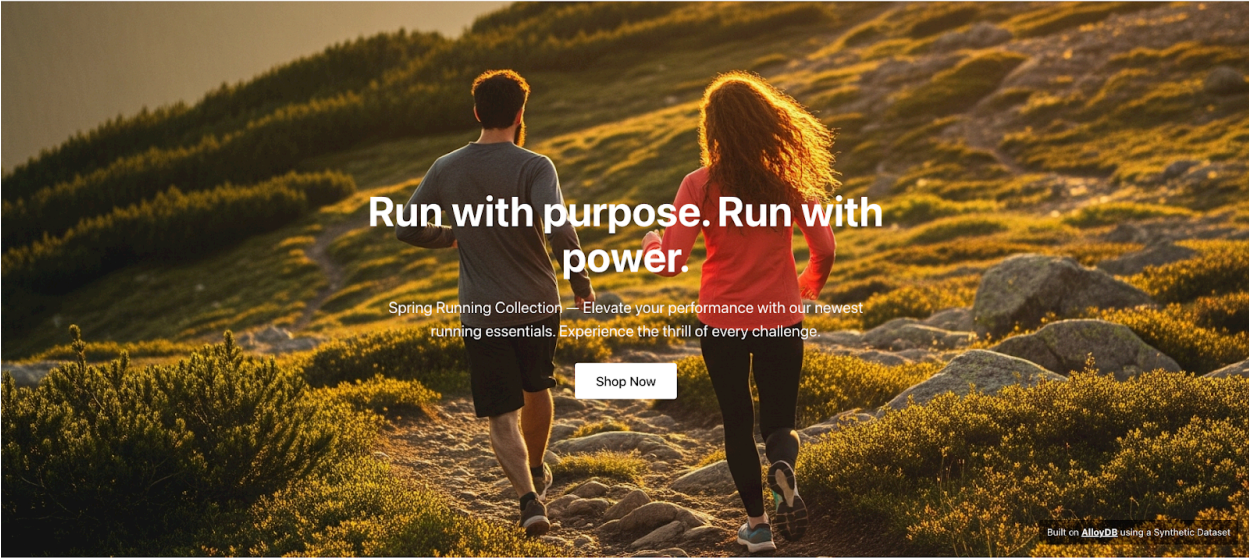
最後，我們將使用下列指令，在 Cloud Run 上部署前端：

```
gcloud run deploy finn-frontend \
  --image us-central1-docker.pkg.dev/$PROJECT_ID/finn-frontend-images/finn-frontend \
  --platform managed \
  --allow-unauthenticated \
  --region us-central1 \
  --project $PROJECT_ID
```

您應該會看到類似以下的輸出內容，表示前端部署作業已順利完成：

```
Deploying container to Cloud Run service [finn-frontend] in project [sport-store-agent-ai]
region [us-central1]
OK Deploying... Done.
  OK Creating Revision...
  OK Routing traffic...
  OK Setting IAM Policy...
Done.
Service [finn-frontend] revision [finn-frontend-00002-mwc] has been deployed and is serving
100 percent of traffic.
Service URL: https://finn-frontend-535807247199.us-central1.run.app
```

開啟網路瀏覽器，然後使用上一步中的服務網址，開啟由 AI 代理程式支援的新部署應用程式！



Run with purpose. Run with power.

Spring Running Collection — Elevate your performance with our newest running essentials. Experience the thrill of every challenge.

Shop Now

Built on AlloyDB using a Synthetic Dataset

步驟 9 · 約 0 分鐘

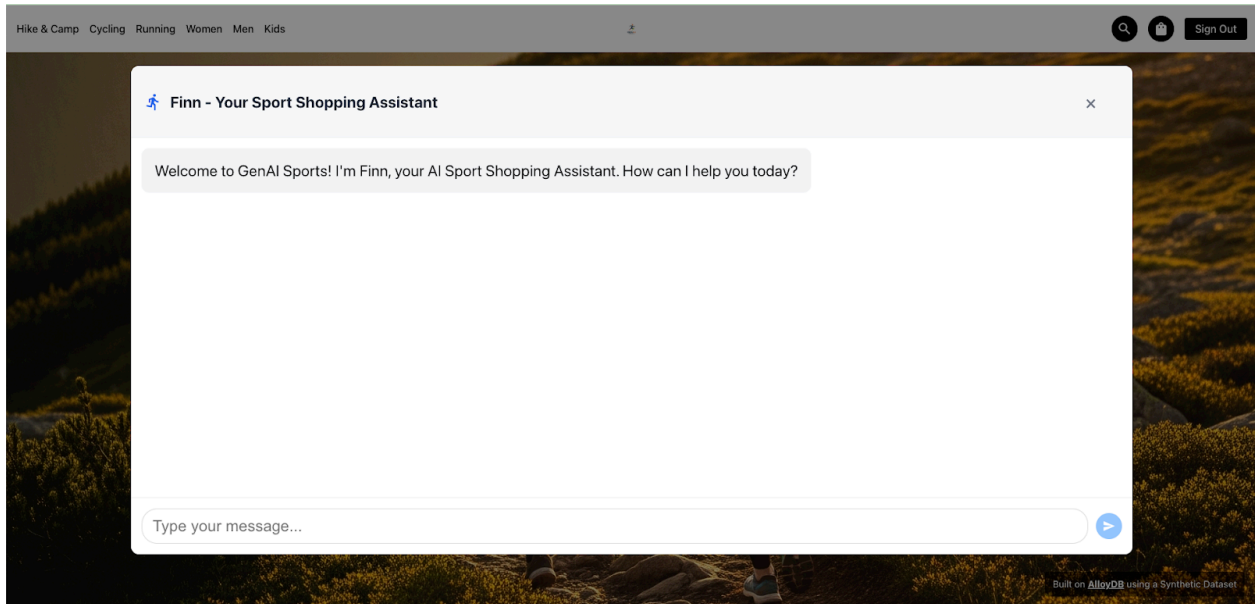
9. 執行代理程式

運動用品店代理 AI 助理 Finn 已全面部署，隨時可協助顧客購物！

開啟網路瀏覽器，然後前往上一步中前端應用程式的服務網址。網址格式如下：`https://finn-frontend-[YOUR_PROJECT_NUMBER].us-central1.run.app`

前端載入後，按一下右上方的按鈕 (通常標示為「登入」或類似提示)，使用 Google 憑證進行驗證。這項操作會使用您先前設定的 OAuth 設定。

驗證成功後，即可與 Finn 互動！按一下「立即購物」按鈕，即可開始對話式購物體驗。



使用下列腳本測試 AI 代理程式的各項功能。將下列提示逐一複製並貼到對話介面：

1. 你好，Finn ！
2. 我想找適合超級越野跑的跑鞋
3. 請提供 Ultra Glide 的詳細資訊
4. 將 Ultra Glide (尺寸 40，顏色為紅/灰) 加入我的購物清單
5. 顯示我的購物清單
6. 尋找附近的商店
7. 請使用我的購物清單，向 Sports Diagonal Mar 下單
8. 查看訂單狀態
9. 請列出「Sports Diagonal Mar」商店的運送方式
10. 將訂單 [YOUR_ORDER_NUMBER] 的運送方式變更為快速運送
11. 查看訂單狀態
12. Thanks Finn ！

若要觀看已部署的 Finn 代理程式及其功能的示範影片，請參閱下方影片：

[AlloyDB 支援的運動代理 AI 助理示範](#)

10. 結果

執行先前的指令碼後，您已成功驗證 ADK 代理的完整整合、與 AlloyDB 的連線，以及 MCP 工具箱的使用情形。本節將重點介紹您實作的核心功能。

1. 授權服務

資料庫適用的 **MCP 工具箱** 可聯合授權服務 (在本程式碼研究室中，具體來說是 Google 登入)，在應用程式中驗證使用者。使用 MCP Toolbox 時，系統會在工具叫用時使用 OAuth 用戶端 ID 驗證使用者身分。

這項強大的驗證機制可有效防範提示詞注入攻擊。這類攻擊會嘗試繞過或操縱代理程式的預期行為，詳情請參閱維基百科上有關[提示注入](#)的文章。

在本應用程式中，當使用者要求「查看訂單狀態」或「顯示購物清單」時，就會使用這項技術。這個代理程式的設計宗旨是只顯示通過驗證的使用者所下的訂單，防止未經授權存取訂單資訊。

```
tools:
  check-order-status-by-user:
    kind: postgres-sql
    source: my-alloydb-pg-source
    description: Check orders status by user id. Return the order_id, total, status and delivery method.
    parameters:
      - name: user_email
        type: string
        description: Auto-populated from Google login
    authServices:
      - name: google_signin
        field: email
```

2. Vector Search

代理式應用程式會運用 **AlloyDB for PostgreSQL** 提供進階查詢功能，特別是向量搜尋。AlloyDB 支援使用 SQL 函式，直接在資料庫中產生線上嵌入。

這項強大功能可讓代理程式將使用者的自然語言輸入內容轉換為數值嵌入表示法。接著，系統會根據這些嵌入內容，對產品目錄 (或其他相關資料) 執行相似度搜尋，提供高度相關的搜尋結果。

在應用程式中，當你詢問 Finn「我想找適合超級越野跑的跑鞋」時，就會體驗到這項技術。

```
search-products-by-description:
  kind: postgres-sql
  source: my-alloydb-pg-source
  description: Search for sport products based on an a user description.
  Return the name, brand, category, description and popularity score of the product.
  parameters:
    - name: description
      type: string
      description: User description of the product they're looking for.
  statement: |
    SELECT
      name,
      description,
      1 - (embedding <=> embedding('text-embedding-005',$1)::vector) as similarity_score
    FROM
      products
    ORDER BY
      similarity_score DESC
    LIMIT 5;
```

3. 地理空間功能 (PostGIS)

AlloyDB for PostgreSQL 完全相容於標準 PostgreSQL 功能。在本應用程式中，我們使用熱門的 PostgreSQL 擴充功能 **PostGIS**，為代理程式提供地理空間位置功能。

當您要求代理程式「尋找我附近的商店」時，代理程式會執行工具，利用資料庫中的 PostGIS 索引，有效找出距離使用者指定或推斷位置最近的商店。

```
find-stores-by-user:
  kind: postgres-sql
  source: my-alloydb-pg-source
  description: Search stores by user ID. First get user location, then find nearby stores.
  parameters:
    - name: user_email
      type: string
      description: Auto-populated from Google login
      authServices:
        - name: google_signin
          field: email
  statement: |
    WITH user_info AS (
      SELECT
        'USER|' ||
        '0,' ||
        ST_X(location::geometry)::numeric(10,6) || ',' ||
        ST_Y(location::geometry)::numeric(10,6) as location_info,
        location as user_location
      FROM users
      WHERE email = $1
    ),
    store_info AS (
      SELECT
        s.name || '|' ||
        ST_Distance(s.location, u.user_location)::integer || ',' ||
        ST_X(s.location::geometry)::numeric(10,6) || ',' ||
        ST_Y(s.location::geometry)::numeric(10,6) as location_info
      FROM stores s, user_info u
      ORDER BY s.location <-> u.user_location
      LIMIT 3
    )
    SELECT string_agg(location_info, E'\n')
    FROM (
      SELECT location_info FROM user_info
      UNION ALL
      SELECT location_info FROM store_info
    ) all_locations,
```

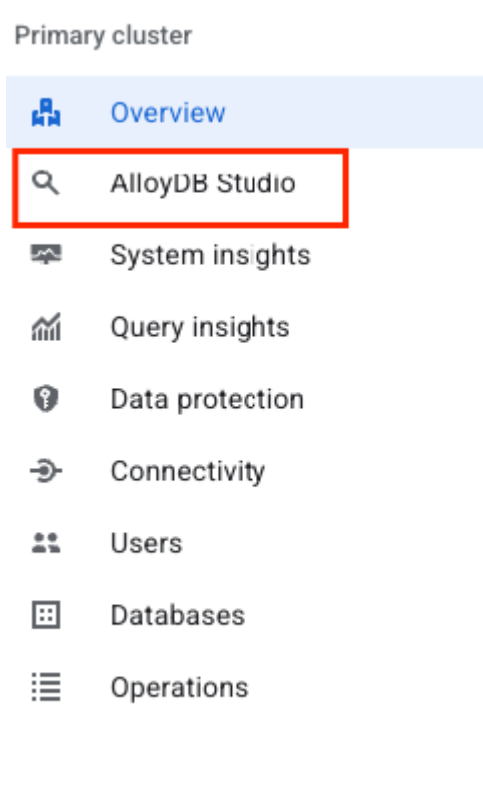
11. (選用) 測試 AlloyDB AI 自然語言轉 SQL 功能

本節將介紹 AlloyDB for PostgreSQL 的進階功能「自然語言轉 SQL」，這項功能目前為 **正式發布前** 階段。這項功能可讓您直接從自然語言提示生成 SQL 查詢，在資料庫中運用 AI 的強大功能。

重要事項：這項功能為正式發布前版本，因此您必須註冊，並為 Google Cloud 專案、AlloyDB 叢集和資料庫啟用存取權。

- **申請存取權：**請填寫這份[表單](#)，為專案申請存取權
- **說明文件：**如要進一步瞭解如何運用 AlloyDB AI 自然語言生成 SQL，請參閱[官方說明文件](#)

註冊並確認專案存取權後，請在 AlloyDB Studio 中完成下列步驟。



使用建立叢集時建立的憑證登入 AlloyDB：

- 使用者名稱：「postgres」
- 資料庫：「store」
- 密碼：「alloydb」

1. 建立 alloydb_ai_nl 擴充功能。這個擴充功能提供 AlloyDB AI 自然語言功能所需的函式。

```
CREATE EXTENSION alloydb_ai_nl cascade;
```

2. 為應用程式建立設定。設定會定義 AI 模型用來瞭解資料庫的結構定義環境。

```
SELECT
  alloydb_ai_nl.g_create_configuration(
    'finn_app_config'      -- configuration_id
  );
```

3. 向設定檔註冊結構定義 / 表格。將應用程式代理程式會互動的特定表格和結構定義新增至設定。

```
SELECT alloydb_ai_nl.g_manage_configuration(  
  operation => 'register_table_view',  
  configuration_id_in => 'finn_app_config',  
  table_views_in=>'{public.products, public.products_variants, public.orders,  
public.orders_items, public.users, public.inventory, public.stores}'  
);
```

4. 為結構定義 / 資料表生成背景資訊。這個步驟會處理已註冊的表格，為 AI 模型生成必要的背景資訊。這個程序大約需要 **2 到 3 分鐘**。

```
SELECT alloydb_ai_nl.generate_schema_context(  
  'finn_app_config',  
  TRUE  
);
```

5. 檢查特定表格和資料欄的自動產生內容 (選用)。您可以檢查生成的內容，瞭解 AI 模型如何解讀結構定義。

```
SELECT object_context  
FROM alloydb_ai_nl.generated_schema_context_view  
WHERE schema_object = 'public.inventory';  
  
SELECT object_context  
FROM alloydb_ai_nl.generated_schema_context_view  
WHERE schema_object = 'public.products.name';  
  
SELECT object_context  
FROM alloydb_ai_nl.generated_schema_context_view  
WHERE schema_object = 'public.products.popularity_score';
```

在 Agent 的「tools.yaml」中，你會看到名為「check-inventory-by-store-brand-category」的工具。這項工具會使用 AlloyDB 自然語言轉 SQL 功能：

```
check-inventory-by-store-brand-category:
  kind: alloydb-ai-nl
  source: my-alloydb-pg-source
  description: "Show the available products by store, brand and category. Return the number of products available"
  nlConfig: "finn_app_config"
  nlConfigParameters:
    - name: user_email
      type: string
      description: User ID of the logged in user.
      authServices:
        - name: google_signin
          field: email
    - name: store_name
      type: string
      description: The name of the store.
    - name: category
      type: string
      description: The category of the product.
    - name: brand
      type: string
      description: The brand of the product.
```

開啟網路瀏覽器，然後使用服務網址開啟應用程式：「[https://finn-frontend-\[YOUR_PROJECT_NUMBER\].us-central1.run.app](https://finn-frontend-[YOUR_PROJECT_NUMBER].us-central1.run.app)」

接著，在即時通訊介面中使用下列指令碼，測試這項新功能：

- 你好，Finn ！
- 在「Sports Diagonal Mar」商店中，Salomon 跑步類別產品的總庫存量有多少現貨？

如要查看 AlloyDB AI 從自然語言輸入內容生成的實際 SQL 查詢，請返回 **AlloyDB Studio** 並執行下列查詢：

```
SELECT
  alloydb_ai_nl.get_sql(
    'finn_app_config',
    'What is the total quantity of category Running products of Salomon in stock at the
    "Sports Diagonal Mar" store?'
  ) ->> 'sql';
```

系統會顯示 AlloyDB AI 生成的 SQL 陳述式。

12. 清除

如要避免系統向您的 Google Cloud 帳戶收取本實驗室所用資源的費用，請按照下列步驟操作：

1. 在 Google Cloud 控制台中前往「管理資源」頁面。
 2. 在專案清單中選取要刪除的專案，然後按一下「刪除」。
 3. 在對話方塊中輸入專案 ID，然後按一下「Shut down」(關閉) 刪除專案。
-

步驟 13 · 約 0 分鐘

13. 恭喜

恭喜！您已成功使用 ADK、MCP Toolbox for Databases 和 AlloyDB for PostgreSQL，建立資料驅動的代理式 AI 應用程式

詳情請參閱產品文件：[Agent Development Kit](#)、[MCP Toolbox for Databases](#) 和 [AlloyDB for PostgreSQL](#)
